

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..48

WRITING TESTS

 Copy as Markdown

Writing tests

Every test you write with the test tools uses the same few pieces. The `withServerpod` callback hands you two of them:

- `sessionBuilder` describes the server state a call runs under, such as who is signed in. You pass it to endpoint calls, and you can build it into a `Session` when you need one directly.
- `endpoints` is your generated endpoints, called the same way your app calls them.

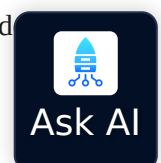
```
withServerpod('Given Greeting endpoint', (sessionBuilder, endpoints) {  
  test('then hello returns a greeting', () async {  
    var greeting = await endpoints.greeting.hello(sessionBuilder, 'Bob');  
    expect(greeting.message, 'Hello Bob');  
  });  
});
```

The generated file carries the test helpers and your endpoints. Serverpod's own types, such as `Session` and `Scope`, and your models come from their own packages:

```
import 'package:serverpod/serverpod.dart';  
import 'package:test/test.dart';  
import 'package:my_project_server/src/generated/protocol.dart';  
  
import 'test_tools/serverpod_test_tools.dart';
```

Set up a scenario

Call `copyWith` on the session builder to change the state a call runs under. It returns a new builder with the original untouched, so each scenario in a test file can have its own.



Property	What it does	If you omit it
<code>authentication</code>	Who is signed in for calls made with this builder.	Inherits from the builder you called <code>copyWith</code> on.
<code>enableLogging</code>	Whether the session writes log entries.	Inherits from the builder you called <code>copyWith</code> on.

A fresh builder from `withServerpod` starts out unauthenticated, and logs only if you passed `enableSessionLogging`.

When you need a `Session` rather than a builder, for a database call or to pass into [your own business logic](#), build one:

```
Session session = sessionBuilder.build();
```

Simulate authentication

The `AuthenticationOverride` class sets who the call runs as. It offers two:

```
static AuthenticationOverride unauthenticated();

static AuthenticationOverride authenticationInfo(
    String userIdentifier,
    Set<Scope> scopes, {
    String? authId,
});
```

The `userIdentifier` is the signed-in user's id, which is a UUID string when you use Serverpod's auth module. Scopes are the permissions the call should carry. See [Authentication](#) for both.

Pass an override through `copyWith` to test each case:

```
withServerpod('Given AuthenticatedExample endpoint', (sessionBuilder, endpoints) {
    final userId = '550e8400-e29b-41d4-a716-446655440000';

    group('when authenticated', () {
        var authenticated = sessionBuilder.copyWith(
            authentication:
                AuthenticationOverride.authenticationInfo(userId, {Scope('user')})),
        );

        test('then calling `hello` returns a greeting', () async {
            final greeting =
```

```

        await endpoints.authenticatedExample.hello(authenticated, 'Michael');
        expect(greeting, 'Hello, Michael!');
    });
});

group('when unauthenticated', () {
    var unauthenticated = sessionBuilder.copyWith(
        authentication: AuthenticationOverride.unauthenticated(),
    );

    test('then calling `hello` throws', () async {
        final future =
            endpoints.authenticatedExample.hello(unauthenticated, 'Michael');
        await expectLater(
            future, throwsA(isA<ServerpodUnauthenticatedException>()));
    });
});
});

```

Seed the database

Build a session and use it for database calls exactly as your server code would:

```

withServerpod('Given Products endpoint', (sessionBuilder, endpoints) {
    var session = sessionBuilder.build();

    setUp(() async {
        await Product.db.insert(session, [
            Product(name: 'Apple', price: 10),
            Product(name: 'Banana', price: 10),
        ]);
    });

    test('then calling `all` returns all products', () async {
        final products = await endpoints.products.all(sessionBuilder);
        expect(products, hasLength(2));
        expect(products.map((p) => p.name), containsAll(['Apple', 'Banana']));
    });
});

```

Everything a test writes to the database is rolled back when the test ends, so each test starts from the same state and you do not have to clean up after yourself. You can change when that happens, or turn it off, with `rollbackDatabase`.

Exceptions the test tools throw

The generated file exports these, so you can name them in a test without importing anything else.

Exception	Thrown when
<code>ServerpodUnauthenticatedException</code>	An endpoint call was made without authentication and the endpoint requires it.
<code>ServerpodInsufficientAccessException</code>	The authentication used did not carry the scopes the endpoint requires.
<code>ConnectionClosedException</code>	A stream connection closed with an error, for example because authentication was revoked.
<code>InvalidConfigurationException</code>	The endpoint ran database calls or transactions in parallel, which the automatic rollback cannot wrap. See <code>rollbackDatabase</code> .

The test server also throws when it fails to start or takes longer than the start timeout. That error type is not exported, so you cannot catch it by name, but the message says which of the two happened.

Wait for async work

The `flushEventQueue` helper lets already-queued async work run before the test carries on. Reach for it when a call has to get as far as its first `yield` before you assert on it:

```
var stream = endpoints.someEndpoint.generatorFunction(sessionBuilder);
await flushEventQueue();
```

For a complete case, see [multiple users interacting with a shared stream](#).

Related

- [Configuration](#): every option `withServerpod` accepts.
- [Advanced examples](#): streams, future calls, and business logic.
- [Sessions](#): what a session carries in production.